S & ASP.NET Monitoring with Datadog

Windows Service Setup Guide — Option B: DogStatsD-CSharp-Client

Audience	Windows Server Administrators / DevOps Engineers
Agent Mode	`infrastructure_mode: basic` (DogStatsD only)
Metrics Covered	11 custom metrics — ASP.NET, IIS, w3wp
Application Changes	None required
SDK	DogStatsD-CSharp-Client v9.0.0 (official Datadog)

Overview

This guide walks through deploying a lightweight .NET Windows Service that collects IIS and ASP.NET performance metrics and sends them to Datadog — without modifying your application code.

When to use this approach: When `infrastructure_mode: basic` is configured in your Datadog Agent. This mode disables all native integration checks (including `windows_performance_counters`, `iis`, and `sqlserver`), but DogStatsD on UDP port 8125 remains active and is the only supported path for custom metric submission.

Architecture

```
IIS / ASP.NET / w3wp.exe
        |
        |  Windows Performance Counters  (OS built-in, no app changes)
        ▼
 DatadogIISMetrics.exe      ← .NET 4.8 Windows Service (this guide)
 reads counters every 15s
        |
        |  UDP  127.0.0.1:8125
        ▼
 Datadog Agent (DogStatsD listener)
        |
        |  HTTPS
        ▼
 Datadog Platform
```

Metrics Collected

Metric	What It Measures

`aspnet.requests.current`	Requests being processed right now
`aspnet.requests.queued`	Requests waiting in the ASP.NET queue
`aspnet.requests.rejected`	Requests rejected due to queue overflow
`aspnet.requests.in_queue`	Requests in the native kernel queue
`aspnet.request.execution_time`	Average request execution time (ms)
`aspnet.request.wait_time`	Average request wait time in queue (ms)
`iis.connections.current`	Active HTTP connections across all IIS sites
`iis.requests.get`	Cumulative total GET requests served
`iis.requests.post`	Cumulative total POST requests served
`w3wp.cpu_pct`	IIS worker process CPU usage (%)
`w3wp.memory_bytes`	IIS worker process memory working set (bytes)

Prerequisites

Requirement	Details
Operating System	Windows Server 2016 / 2019 / 2022
IIS	Version 8.5 or later with ASP.NET 4.x application pool
Datadog Agent	Version 7, installed and running
PowerShell	Version 5.1 or later — **run all commands as Administrator**
Internet access	To download the NuGet package (or pre-download offline)

Step 1 — Verify DogStatsD is Listening

Open **PowerShell as Administrator** and run:

```
netstat -an | findstr ":8125"
```

Expected output:

```
UDP    127.0.0.1:8125    *:*
```

> If port 8125 is not listed, check `C:\ProgramData\Datadog\datadog.yaml`. DogStatsD is enabled by default. If explicitly disabled, add `use_dogstatsd: true` and restart the Datadog Agent service before continuing.

Step 2 — Create the Working Directory

```
New-Item -ItemType Directory -Path "C:\DatadogMetrics" -Force
```

All files for this solution are stored in `C:\DatadogMetrics\`.

Step 3 — Download the StatsdClient DLL

The official `DogStatsD-CSharp-Client` NuGet package contains the required DLL. A `.nupkg` file is a zip archive and can be extracted without any NuGet tooling.

```
# Download the NuGet package
$url  = "https://www.nuget.org/api/v2/package/DogStatsD-CSharp-Client/9.0.0"
$pkg  = "C:\DatadogMetrics\dogstatsd.nupkg"
Invoke-WebRequest -Uri $url -OutFile $pkg

# Extract the .NET 4.6.1 DLL (compatible with .NET Framework 4.8)
Add-Type -AssemblyName System.IO.Compression.FileSystem
$zip   = [System.IO.Compression.ZipFile]::OpenRead($pkg)
$entry = $zip.Entries |
         Where-Object { $_.FullName -like "*/net461/StatsdClient.dll" } |
         Select-Object -First 1
[System.IO.Compression.ZipFileExtensions]::ExtractToFile(
    $entry, "C:\DatadogMetrics\StatsdClient.dll", $true)
$zip.Dispose()

Write-Host "DLL size: $((Get-Item 'C:\DatadogMetrics\StatsdClient.dll').Length) bytes"
# Expected: 67072 bytes
```

> **Offline / Air-gapped servers:** Download the `.nupkg` on a machine with internet access, rename it to `.zip`, open it, copy `lib/net461/StatsdClient.dll` to `C:\DatadogMetrics\StatsdClient.dll` on the target server.

Step 4 — Create the C# Source File

Run the block below in PowerShell. **Edit the `Tags` array** on the lines marked `CUSTOMIZE` before running.

```
$src = @'
using System;
using System.Diagnostics;
using System.ServiceProcess;
using System.Threading;
using StatsdClient;

public class IISMetricsService : ServiceBase
{
    private Thread           _thread;
    private volatile bool    _running;
    private DogStatsdService _statsd;

    // ── CUSTOMIZE ──────────────────────────────────────────────
    private static readonly string[] Tags = {
        "env:production",   // CUSTOMIZE: your environment
        "app:my-app",       // CUSTOMIZE: your application name
        "tech:aspnet",
        "tech:iis"
    };
    // ──────────────────────────────────────────────────────────

    private const string SVC_NAME = "DatadogIISMetrics";
    private const string EVT_SRC  = "DatadogIISMetrics";
    private const int    INTERVAL = 15000;

    public IISMetricsService() { ServiceName = SVC_NAME; }
    static void Main() { ServiceBase.Run(new IISMetricsService()); }

    protected override void OnStart(string[] args)
    {
        EnsureEventSource();
        _statsd = new DogStatsdService();
        _statsd.Configure(new StatsdConfig {
            StatsdServerName = "127.0.0.1", StatsdPort = 8125
        });
        _running = true;
        _thread  = new Thread(RunLoop) { IsBackground = true };
        _thread.Start();
        EventLog.WriteEntry(EVT_SRC,
            "DatadogIISMetrics started.", EventLogEntryType.Information, 1);
    }

    protected override void OnStop()
    {
        _running = false;
        if (_statsd != null) { _statsd.Dispose(); }
    }
```

```
    private void RunLoop()
    {
        while (_running)
        {
            try   { Collect(); }
            catch (Exception ex)
            {
                EventLog.WriteEntry(EVT_SRC,
                    "Collect() error: " + ex.Message,
                    EventLogEntryType.Error, 10);
            }
            Thread.Sleep(INTERVAL);
        }
    }

    private void Collect()
    {
        Gauge("aspnet.requests.current",
              "ASP.NET v4.0.30319", "Requests Current",        null);
        Gauge("aspnet.requests.queued",
              "ASP.NET v4.0.30319", "Requests Queued",          null);
        Gauge("aspnet.requests.rejected",
              "ASP.NET v4.0.30319", "Requests Rejected",        null);
        Gauge("aspnet.request.execution_time",
              "ASP.NET v4.0.30319", "Request Execution Time",   null);
        Gauge("aspnet.request.wait_time",
              "ASP.NET v4.0.30319", "Request Wait Time",        null);
        Gauge("aspnet.requests.in_queue",
              "ASP.NET v4.0.30319", "Requests In Native Queue", null);
        Gauge("iis.connections.current",
              "Web Service", "Current Connections",  "_Total");
        Gauge("iis.requests.get",
              "Web Service", "Total Get Requests",   "_Total");
        Gauge("iis.requests.post",
              "Web Service", "Total Post Requests",  "_Total");
        GaugeW3wp("w3wp.cpu_pct",      "Process", "% Processor Time");
        GaugeW3wp("w3wp.memory_bytes", "Process", "Working Set");
    }

    private void Gauge(string metric, string category,
                       string counter, string instance)
    {
        try
        {
            PerformanceCounter pc = (instance != null)
                ? new PerformanceCounter(category, counter, instance, true)
                : new PerformanceCounter(category, counter, true);
            _statsd.Gauge(metric, pc.NextValue(), tags: Tags);
        }
        catch (Exception ex)
        {
            EventLog.WriteEntry(EVT_SRC,
                "Gauge failed [" + metric + "]: " + ex.Message,
                EventLogEntryType.Warning, 20);
        }
    }

    private void GaugeW3wp(string metric, string category, string counter)
    {
        try
        {
            double sum = 0; bool found = false;
            for (int i = 0; i < 8; i++)
            {
```

```
                string inst = (i == 0) ? "w3wp" : "w3wp#" + i;
                try
                {
                    if (!PerformanceCounterCategory.InstanceExists(
                            inst, category)) break;
                    sum  += new PerformanceCounter(
                                category, counter, inst, true).NextValue();
                    found = true;
                }
                catch (Exception) { break; }
            }
            if (found) _statsd.Gauge(metric, sum, tags: Tags);
        }
        catch (Exception ex)
        {
            EventLog.WriteEntry(EVT_SRC,
                "GaugeW3wp failed [" + metric + "]: " + ex.Message,
                EventLogEntryType.Error, 40);
        }
    }

    private void EnsureEventSource()
    {
        try
        {
            if (!EventLog.SourceExists(EVT_SRC))
                EventLog.CreateEventSource(EVT_SRC, "Application");
        }
        catch (Exception) { }
    }
}
'@
[System.IO.File]::WriteAllText(
    "C:\DatadogMetrics\IISMetricsService.cs", $src)
Write-Host "Source file written."
```

Step 5 — Compile the Service

Windows Server ships with the .NET Framework C# compiler (`csc.exe`). **No Visual Studio or SDK is required.**

```
$csc = "C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe"

& $csc /target:exe `
       /out:C:\DatadogMetrics\DatadogIISMetrics.exe `
       /reference:C:\DatadogMetrics\StatsdClient.dll `
       /reference:C:\Windows\Microsoft.NET\Framework64\v4.0.30319\System.ServiceProcess.dll `
       /reference:C:\Windows\Microsoft.NET\Framework64\v4.0.30319\System.dll `
       C:\DatadogMetrics\IISMetricsService.cs

Write-Host "EXE size: $((Get-Item 'C:\DatadogMetrics\DatadogIISMetrics.exe').Length) bytes"
# Expected: ~8192 bytes
```

A successful compile shows **only the compiler version banner** — no error lines. If you see errors, refer to the Troubleshooting section.

> **Important:** `csc.exe` from `Framework64\v4.0.30319` supports **C# 5 only**. Do not use `?.` (null-conditional), `$"..."` (string interpolation), or `nameof()` . The source code in Step 4 is fully C# 5 compatible.

Step 6 — Install the Windows Service

```
# Register the service
sc.exe create DatadogIISMetrics `
    binPath= "C:\DatadogMetrics\DatadogIISMetrics.exe" `
    start=   auto `
    obj=     LocalSystem `
    DisplayName= "Datadog IIS Metrics"

# Add a description
sc.exe description DatadogIISMetrics `
    "Sends IIS and ASP.NET Performance Counter metrics to Datadog via DogStatsD"

# Start the service
Start-Service DatadogIISMetrics

# Confirm it is running
Get-Service DatadogIISMetrics | Select-Object Name, Status
```

Expected output:

```
Name               Status
----               ------
DatadogIISMetrics Running
```

The service is configured for **automatic startup** and will restart automatically after a server reboot.

Step 7 — Confirm the Service Started

Check the Windows Application Event Log for the startup message:

```
Get-EventLog -LogName Application -Source DatadogIISMetrics -Newest 5 |
    Select-Object TimeGenerated, EntryType, Message |
    Format-List
```

Expected output:

```
TimeGenerated : 4/12/2026 11:53:43 AM
EntryType     : Information
Message       : DatadogIISMetrics started.
```

Step 8 — Validate Metrics in Datadog

Open **Metrics Explorer** (`app.datadoghq.com/metric/explorer`). All 11 metrics should appear within **30–60 seconds** of the service starting.

Recommended validation queries:

```
avg:aspnet.requests.current{*}      avg:iis.connections.current{*}
avg:aspnet.requests.queued{*}       avg:iis.requests.get{*}
avg:aspnet.request.execution_time{*} avg:w3wp.cpu_pct{*}
avg:aspnet.request.wait_time{*}     avg:w3wp.memory_bytes{*}
```

Optional — send a one-off test metric to confirm connectivity:

```
$udp = New-Object System.Net.Sockets.UdpClient
$udp.Connect("127.0.0.1", 8125)
$b = [System.Text.Encoding]::UTF8.GetBytes("iis.healthcheck:1|g|#env:production")
$udp.Send($b, $b.Length) | Out-Null
$udp.Close()
Write-Host "Look for 'iis.healthcheck' in Datadog Metrics Explorer."
```

Day-2 Operations

Stop, Start, and Restart

```
Stop-Service    DatadogIISMetrics -Force
Start-Service   DatadogIISMetrics
Restart-Service DatadogIISMetrics
```

View Service Logs

```
Get-EventLog -LogName Application -Source DatadogIISMetrics -Newest 20 | Format-List
```

Update Tags or Add Metrics (Recompile)

```
# 1. Stop
Stop-Service DatadogIISMetrics -Force

# 2. Edit  →  notepad C:\DatadogMetrics\IISMetricsService.cs

# 3. Recompile
$csc = "C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe"
& $csc /target:exe `
       /out:C:\DatadogMetrics\DatadogIISMetrics.exe `
       /reference:C:\DatadogMetrics\StatsdClient.dll `
       /reference:C:\Windows\Microsoft.NET\Framework64\v4.0.30319\System.ServiceProcess.dll `
       /reference:C:\Windows\Microsoft.NET\Framework64\v4.0.30319\System.dll `
       C:\DatadogMetrics\IISMetricsService.cs

# 4. Start
Start-Service DatadogIISMetrics
```

Uninstall

```
Stop-Service DatadogIISMetrics -Force
sc.exe delete DatadogIISMetrics
Remove-Item "C:\DatadogMetrics" -Recurse -Force
```

Troubleshooting

No Metrics in Datadog After 2 Minutes

1. Confirm DogStatsD is listening

```
netstat -an | findstr ":8125"
# Must show: UDP    127.0.0.1:8125    *:*
```

If not shown: `Restart-Service datadogagent`

2. Confirm the service is running

```
Get-Service DatadogIISMetrics
```

3. Send a test metric and search in Metrics Explorer

```
$udp = New-Object System.Net.Sockets.UdpClient
$udp.Connect("127.0.0.1", 8125)
$b = [System.Text.Encoding]::UTF8.GetBytes("iis.test:42|g|#env:production")
$udp.Send($b, $b.Length) | Out-Null; $udp.Close()
```

---

w3wp.cpu_pct and w3wp.memory_bytes Show No Data

The IIS application pool shuts down `w3wp.exe` after **20 minutes of no traffic** (IIS idle timeout default). No w3wp process means no Performance Counter instances.

Immediate fix: send requests to restart the worker:

```
1..10 | ForEach-Object {
    Invoke-WebRequest http://localhost/ -UseBasicParsing | Out-Null
}
```

Permanent fix: disable the idle timeout in IIS Manager: `Application Pools → [your pool] → Advanced Settings → Idle Time-out (minutes) → 0`

Compile Errors

Error Message	Cause	Fix
`error CS1525: Invalid expression term '.'`	Using `?.` null-conditional (C# 6+)	Use `if (x ≠ null) { x.Method(); }` instead
`Metadata file 'StatsdClient.dll' could not be found`	DLL missing	Re-run Step 3

`The system cannot find the path specified`	Wrong `csc.exe` path	Run: `Test-Path "C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe"`
`The type or namespace 'StatsdClient' could not be found`	Missing `/reference:`	Add `/reference:C:\DatadogMetrics\StatsdClient.dll`

Windows Event Log Error Codes

Event ID	Type	Meaning
1	Information	Service started successfully
10	Error	Unhandled exception in `Collect()` — check Message for details
20	Warning	A single counter could not be read — others continue working
40	Error	`GaugeW3wp` failed — likely a permissions issue on Process counters

Service Won't Start (Error 1053)

Check that both files exist and were compiled together:

- `C:\DatadogMetrics\DatadogIISMetrics.exe`
- `C:\DatadogMetrics\StatsdClient.dll`

If in doubt, re-run Steps 5 and 6.

Quick Reference — All Commands in Order

```
# STEP 1 — Verify DogStatsD
netstat -an | findstr ":8125"

# STEP 2 — Create directory
New-Item -ItemType Directory -Path "C:\DatadogMetrics" -Force

# STEP 3 — Download StatsdClient.dll
$url = "https://www.nuget.org/api/v2/package/DogStatsD-CSharp-Client/9.0.0"
Invoke-WebRequest -Uri $url -OutFile "C:\DatadogMetrics\dogstatsd.nupkg"
Add-Type -AssemblyName System.IO.Compression.FileSystem
$zip = [System.IO.Compression.ZipFile]::OpenRead(
           "C:\DatadogMetrics\dogstatsd.nupkg")
$entry = $zip.Entries |
         Where-Object { $_.FullName -like "*/net461/StatsdClient.dll" } |
         Select-Object -First 1
[System.IO.Compression.ZipFileExtensions]::ExtractToFile(
    $entry, "C:\DatadogMetrics\StatsdClient.dll", $true)
$zip.Dispose()

# STEP 4 — Write source  →  run the $src block from Step 4

# STEP 5 — Compile
$csc = "C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe"
& $csc /target:exe /out:C:\DatadogMetrics\DatadogIISMetrics.exe `
    /reference:C:\DatadogMetrics\StatsdClient.dll `
    /reference:C:\Windows\Microsoft.NET\Framework64\v4.0.30319\System.ServiceProcess.dll `
    /reference:C:\Windows\Microsoft.NET\Framework64\v4.0.30319\System.dll `
    C:\DatadogMetrics\IISMetricsService.cs

# STEP 6 — Install and start
sc.exe create DatadogIISMetrics `
    binPath= "C:\DatadogMetrics\DatadogIISMetrics.exe" `
    start= auto obj= LocalSystem DisplayName= "Datadog IIS Metrics"
Start-Service DatadogIISMetrics

# STEP 7 — Verify
Get-Service DatadogIISMetrics
Get-EventLog -LogName Application -Source DatadogIISMetrics -Newest 3 | Format-List

# STEP 8 — Open Metrics Explorer
#  avg:aspnet.requests.current{*}
#  avg:iis.requests.get{*}
#  avg:w3wp.memory_bytes{*}
```

Files Reference

Path	Purpose
`C:\DatadogMetrics\IISMetricsService.cs`	C# source — edit to customize tags or add metrics
`C:\DatadogMetrics\StatsdClient.dll`	DogStatsD-CSharp-Client v9.0.0 (net461)

`C:\DatadogMetrics\DatadogIISMetrics.exe`	Compiled Windows Service binary
`C:\ProgramData\Datadog\datadog.yaml`	Datadog Agent config (not modified by this guide)

Datadog Agent 7 · Windows Server 2016/2019/2022 · .NET Framework 4.8 · DogStatsD-CSharp-Client v9.0.0 · infrastructure_mode: basic